

```

n_tup_upd      | 200030
n_tup_hot_upd  | 106
n_live_tup     | 100000
n_dead_tup     | 50000
last_vacuum    |
last_autovacuum | 2023-09-15 15:13:47.424223+00
last_analyze   |
last_autoanalyze | 2023-09-15 15:13:47.60569+00

```

The `last_vacuum`, `last_analyze`, `last_autovacuum`, and `last_autoanalyze` columns are particularly important to understand whether manual or automatic vacuuming and analysis ran on the table; this knowledge can be crucial to understanding whether the automatic daemons are working properly. The `n_live_tup` column reports the currently visible tuples, according to MVCC (see *Chapter 11, Transactions, MVCC, WALs, and Checkpoints*), while the `n_dead_tup` column reports the number of no longer visible tuples that still occupy space but will be reclaimed by a manual or automatic vacuum.

The other columns are pretty much self-explanatory, with `seq_scan` and `idx_scan` being the number of times the table has been accessed in a sequential scan or by an index among those available; `n_tup_ins`, `n_tup_upd`, and `n_tup_del` provide information about how many tuples have been inserted as new and how many have been updated or deleted, respectively. The `n_tup_upd_hot` column reports the number of tuples that have been updated in place, instead of being created as new, by means of a mechanism called **Heap Only Tuple (HOT)**.

The `pg_stat_user_indexes` special catalog provides detailed information about the usage of the available indexes. In particular, the `idx_scan`, `idx_tup_read`, and `idx_tup_fetch` fields specify the number of times the index has been used, how many index tuples have been read, and how many table tuples have been obtained thanks to the index. For more information, please see *Chapter 13, Query Tuning, Indexes, and Performance Optimization*.

There are other, dual, catalogs whose names include “all” or “sys” to indicate they refer to all the available tables, including PostgreSQL internal tables, or to only the latter (system tables). Therefore, `pg_stat_all_tables` is the same as `pg_stat_user_tables` but also includes information about system tables, which is kept under `pg_stat_sys_tables`. The same applies to `pg_stat_all_indexes`; that is, the union of `pg_stat_user_indexes` and `pg_stat_sys_indexes`.

More statistics

PostgreSQL includes a very rich set of statistics-related catalogs, and not all of them can be described here due to space limitations.

Some of the most important ones to mention include the following:

- `pg_stat_replication`, `pg_stat_replication_slots`, `pg_stat_wal_receiver`, and `pg_stat_subscription` gather information about the replica status of the cluster.
- `pg_stat_bgwriter` gets information about input/output.
- `pg_stat_archiver` gets information about how WALs are being archived.
- `pg_statio_user_tables`, `pg_statio_user_indexes`, and the related `pg_statio_all_tables` and `pg_statio_all_indexes` provide information about input/output at a table or index level, indicating the number of hits and misses from the buffer cache and reading new pages from storage.
- `pg_stat_database` and `pg_stat_database_conflicts` provide information about the status of a database, including executed transactions, conflicts, rollbacks, and so on.

There are also a lot of *progress* statistics that show records only for ongoing operations and their progress status. The progress statistics you're most likely to want to use are:

- `pg_stat_progress_analyze` and `pg_stat_progress_vacuum` provide information about any ANALYZE or VACUUM operation, respectively.
- `pg_stat_progress_cluster` provides information about the progress of any CLUSTER or VACUUM FULL operation.
- `pg_stat_progress_copy` provides information about any COPY command, thus being useful also for `pg_dump`-related activities.
- `pg_stat_progress_create_index` shows how an index creation is performing.
- `pg_stat_progress_basebackup` shows information about a base backup, a physical way to copy a running cluster.

You should take the time to become comfortable with all the statistics catalogs in order to be able to monitor your cluster with confidence.

In the next section, you are going to learn about a very handy extension that can help you manage your cluster and take control of cluster activities.

Advanced statistics with `pg_stat_statements`

While the PostgreSQL statistics collector is rich and mature, having to monitor connection activity can be a little tricky since the `pg_stat_activity` catalog does not provide historical information. For example, as we explained previously, there will be a single tuple with the last executed statement, so no history nor extended details will be provided.

The `pg_stat_statements` extension solves this problem by providing a single view that gives you a full history of executed statements, timing, and other little details that can come in very handy when doing introspection. Moreover, `pg_stat_statements` provides a count of how many times the same statement has been executed, resulting in important information that queries might need to pay attention to for optimization purposes.

Several monitoring tools require `pg_stat_statements` to be installed in order to gather data.

In the following subsections, you will learn how to install this extension and use it.

Installing the `pg_stat_statements` extension

This extension is shipped with PostgreSQL, so the only thing you have to do is configure the database cluster to use it. Since `pg_stat_statements` requires a shared library, you need to configure the `shared_preload_libraries` setting of your configuration (the `postgresql.conf` file) and restart the cluster.

The first step is to set the following in `postgresql.conf`:

```
shared_preload_libraries = 'pg_stat_statements'
```

Or use an `ALTER SYSTEM` like this:

```
ALTER SYSTEM SET shared_preload_libraries to 'pg_stat_statements';
```

As `pg_shared_preload_libraries` is a parameter with the context `postmaster`, you need to restart the cluster in order to apply the changes.

`pg_stat_statements` collects information about all your clusters, but it will only export such information in the database you create the extension within, which, in our example, is the `forumdb` database:

```
$ psql -U postgres -c "CREATE EXTENSION pg_stat_statements;" forumdb
CREATE EXTENSION
```

The extension is now ready to be used.



In the Docker image for this chapter, the `pg_stat_statements` extension has already been installed and loaded into the `forumdb` database.

Using `pg_stat_statements`

Once `pg_stat_statements` has been enabled, it will start collecting information. The runtime overhead of the extension is really minimal, so you can keep it enabled in production systems too.

Since `pg_stat_statements` collects data from the whole cluster, it is helpful to join the `pg_stat_statements` special view with other catalogs, such as `pg_database` and `pg_authid`, to gather information about the database and username a statement has been executed inside of, respectively. The following query provides an example of this:

```
forumdb=# SELECT auth.rolname, query, db.datname, calls, min_exec_time,
max_exec_time
FROM pg_stat_statements
      JOIN pg_authid auth ON auth.oid = userid
      JOIN pg_database db ON db.oid = dbid
ORDER BY calls DESC;
...
rolname      | postgres
query        | SELECT count(*) FROM forum.posts WHERE last_edited_on >=
CURRENT_DATE - $1
datname      | forumdb
calls        | 17
min_exec_time | 0.037292
max_exec_time | 0.04165
```

The preceding example shows that the query has been executed 17 times since `pg_stat_statements` started collecting the data, and it required between 0.037 to 0.042 milliseconds to run. Depending on the frequency and timing of each query, it could be interesting to inspect and optimize the query by means of an index.

In the above example, the query is reported as a *normalized query*: every parameter, even if literal, has been removed and substituted by a placemark `$1` (other parameters will be marked as `$2`, `$3`, and so on): `SELECT count(*) FROM forum.posts WHERE last_edited_on >= CURRENT_DATE - $1.`

The idea is to track a group of queries that have the same normalized text so that you can get an idea of how many times such a group has been executed, even if with different arguments.

The `pg_stat_statements` special view keeps track of the most frequently executed queries up to the value of the configuration parameter `pg_stat_statements.max`, which defaults to 5000. Once the limit is reached, the least executed queries will be discarded in favor of fresh new ones. This ensures that the space occupied by the `pg_stat_statements` table will remain pretty much constant without any regard to the number of executed statements.

The `pg_stat_statements` view provides many fields that cannot be discussed in detail here, ranging from planning time to buffers and I/O activity. This extension is very useful when you want to deal with the workload of your cluster.

Resetting data collected from `pg_stat_statements`

It is possible, at any given time, to reset all the data that's been collected by the extension that's invoking the `pg_stat_statements_reset()` function as a database administrator. The function will erase all the data that's been collected and will allow the extension to collect new data from scratch. This can be useful when you want to test new configurations or hardware without having the collected data be biased due to old statistics:

```
forumdb=# SELECT pg_stat_statements_reset();
```

By default, `pg_stat_statements` data is kept across clean database shutdowns and restarts.

Tuning `pg_stat_statements`

The extension allows database administrators to limit the amount of data that's collected. In particular, you can tune the following parameters in your `postgresql.conf` configuration file:

- `pg_stat_statements.max` indicates the maximum number of individual queries to collect.
- `pg_stat_statements.save` is a Boolean that indicates whether the content of the collected data must survive a clean system reboot. By default, this setting is true.
- `pg_stat_statements.track` allows you to specify the nesting level to track. With the top value, the extension will collect data about the query that was issued directly within clients and within tracking nested statements. This is triggered by the execution of other statements (for example, in function statements). With the value of `all`, the extension will trigger every statement and its descendants, while with `none`, no data will be collected about user statements.

- `pg_stat_statements.track_utility` tracks all statements that are not in `SELECT`, `INSERT`, `UPDATE`, or `DELETE` – in other words, “non-ordinary” statements. By default, this setting is on.

Usually, you don’t have to tune these settings since `pg_stat_statements` comes already configured to track what most use cases need.

Summary

In this chapter, you learned how PostgreSQL manages configuration through a main text file, `postgresql.conf`, that can be split into smaller files, including the automatically loaded `postgresql.auto.conf`, which is always loaded at the end of the configuration process. Every configuration option can be edited in the configuration file and can be inspected within the database thanks to dedicated system catalogs. This allows the database administrator to not only have a clear understanding of the currently running configuration but to also search for configuration errors and incorrectly loaded settings.

PostgreSQL also collects *statistics*; that is, runtime data that was gathered during the cluster’s operational time. Those statistics can help an administrator understand what is going on, or what happened in the near past, in the cluster. Thanks to a different set of catalogs, which was exposed in this chapter, you learned how to dig into the details of all the information that PostgreSQL has collected for you. Being able to track and analyze what single applications, users, and connections are doing in a specific moment against the cluster provides database administrators with a great way to fix bottlenecks and other problems, thus helping to improve the cluster experience.

Finally, you learned about the `pg_stat_statements` extension, thanks to which it is possible to collect historical data about query execution and timing so that it is possible to apply optimization and deep analysis of the cluster activity.

Now that you’ve understood how to configure and monitor your cluster, it is time to learn how to replicate this. The next chapter will show you how to perform physical replication by configuring the cluster appropriately.

Verify your knowledge

- What is a configuration context?

A configuration context defines how the cluster will perceive changes to a configuration parameter – for example, only at boot time or at the next incoming connection. See the *Configuration contexts* section for more details.

- What is the difference between the catalogs `pg_settings` and `pg_file_settings`?

The `pg_settings` catalog shows the values of every configuration parameter, as well as its admitted and valid values; the `pg_file_settings` catalog shows where (i.e., in which file and at which line) a configuration parameter has been found and loaded. See the *Inspecting all configuration parameters* section for more details.

- Besides editing configuration files, how can you modify the cluster configuration via SQL statements?

You can issue an `ALTER STATEMENT` command to change the values of a configuration setting. Changes will be written into the `postgresql.auto.conf` file. See the *Modifying the configuration from a live system* section for more details.

- How can you get information about running connections, transactions, and queries?

The special catalog `pg_stat_activity` provides information about every single backend process, its running (or last run) query, and its transaction state. See the *Information about running queries and connections* section for more details.

- What does the `pg_stat_statements` extension do?

The `pg_stat_statements` extension provides a historical view of the most commonly repeated queries, with information about the running times, the number of executions, and other details. See the *Advanced statistics with pg_stat_statements* section for more details.

References

- PostgreSQL cluster configuration, official documentation: <https://www.postgresql.org/docs/current/runtime-config.html>
- PGConfig online configurator: <https://www.pgconfig.org/>
- PostgreSQL statistics collector official documentation: <https://www.postgresql.org/docs/current/monitoring-stats.html>
- PostgreSQL `pg_stat_statements` official documentation: <https://www.postgresql.org/docs/current/pgstatstatements.html>

Learn more on Discord

To join the Discord community for this book – where you can share feedback, ask questions to the author, and learn about new releases – follow the QR code below:

<https://discord.gg/jYWCjF6Tku>



17

Physical Replication

When a database, after passing the development and testing phases, arrives in production, the first problem that the DBA must address is managing replicas. Replicas must be managed in real time and automatically updated. Replicas allow us to always have a copy of our data updated in real time on another machine. This machine can be placed in the same data center as our data or in a different one. This chapter differs from all that we have seen previously in that we will talk about physical replication. In PostgreSQL, starting from version 9.x, it is possible to have physical replication natively. We will talk about what physical replication means, and we will see how to create a replica server and how to manage it. We will also see that it is possible to have synchronous or asynchronous replicas and that there can be multiple replicas of the same database, as well as the possibility of having replicas in a cascade.

In this chapter, we will return to the topic of WAL, something we have already discussed in *Chapter 11, Transactions, MVCC, WAL, and Checkpoints*. In order to execute the commands that will be shown in this chapter, we need to install a PostgreSQL server on two machines, or install two instances of PostgreSQL on the same machine but running on different ports. In the rest of the chapter, it will be presumed that you have two PostgreSQL installations available on different machines, to better simulate the situation of a real production environment, and starting from this chapter, we will learn how to install and configure a physical replication.